```
#lab 2 ---> Machine Learning Techniques for Cyber Security
# ENSP355 | AI in Cyber Security
# Akansh Pandey 2401830006 Bsc(Hons)CyberSecurity
```

```
# Practical 1: Supervised ML for Intrusion Detection
# Dataset: NSL-KDD (KDDTrain+.txt); this notebook creates an 80:20 stratified sp

import io, urllib.request, pandas as pd, numpy as np, matplotlib.pyplot as plt,
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, accuracy_score, confusion_mat
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
from sklearn.base import clone
from time import perf_counter

COLUMNS = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_byt

# Public GitHub mirror of the NSL-KDD text file. If this mirror is unavailable,
# download KDDTrain+.TXT from an authorized NSL-KDD mirror and upload it to the
URL = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain%2B.txt
try:
    raw = urllib.request.urlopen(URL, timeout=30).read()
    df = pd.read_csv(io.BytesIO(raw), header=None, names=COLUMNS)
except Exception as exc:
    raise RuntimeError("Dataset download failed. Upload KDDTrain+.TXT and read i

print("Dataset shape:", df.shape)
display(df.head())
print("\nMissing values:", int(df.isna().sum().sum()))
print("\nLabel distribution:")
display(df["label"].value_counts().to_frame("records"))

# Binary target: normal versus attack. This avoids an extremely sparse multiclas
# while preserving the core intrusion-detection task.
df["target"] = (df["label"].astype(str).str.strip().str.lower() != "normal").ast
plt.figure(figsize=(6,4))
df["target"].map({0:"Normal",1:"Attack"}).value_counts().plot(kind="bar")
plt.title("NSL-KDD Class Distribution")
plt.ylabel("Number of records")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Remove target-related columns from predictors. "difficulty" is dataset metadat
X = df.drop(columns=["label", "difficulty", "target"], errors="ignore")
y = df["target"]
categorical = X.select_dtypes(include=["object"]).columns.tolist()
numeric = X.select_dtypes(exclude=["object"]).columns.tolist()
print("Number of input features:", X.shape[1])
print("Categorical features:", categorical)
print("Numeric features:", len(numeric))

# Required 80:20 train-test split. Stratification keeps class proportions simila
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.20, random_state=42, stratify=y
)

# One-hot encoding is suitable for nominal categories (protocol_type, service, f
# Standardization is important for SVM; the same preprocessing is kept in each p
preprocess = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical),
    ("numeric", StandardScaler(), numeric)
])

models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42, class_weight="balan
    "Random Forest": RandomForestClassifier(n_estimators=120, random_state=42,
                                            n_jobs=-1, class_weight="balanced_s
    "SVM (linear)": LinearSVC(random_state=42, class_weight="balanced", max_iter
}
results = []
fitted = {}
for name, model in models.items():
    pipe = Pipeline([("preprocess", clone(preprocess)), ("model", model)])
    start = perf_counter()
    pipe.fit(X_train, y_train)
    elapsed = perf_counter() - start
    pred = pipe.predict(X_test)
    fitted[name] = pipe
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Training time (s)": round(elapsed, 2)
    })
    print(f"\n{name}\n")
    print(classification_report(y_test, pred, target_names=["Normal", "Attack"],
    ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=["Norma
                                            cmap="Blues", values_format="d")
    plt.title(f"Confusion Matrix: {name}")
    plt.tight_layout()
    plt.show()

results_df = pd.DataFrame(results).sort_values("Accuracy", ascending=False)
display(results_df)

# Interpretation reminder:
# Accuracy alone can hide missed attacks. Discuss attack-class recall and F1-sco
print("Best accuracy in this run:", results_df.iloc[0]["Model"])
print("Use the measured results above in your report; do not copy hypothetical s
```

```
Dataset shape: (125973, 43)
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragme
0	0	tcp	ftp_data	SF	491	0	0	
1	0	udp	other	SF	146	0	0	
2	0	tcp	private	S0	0	0	0	
3	0	tcp	http	SF	232	8153	0	
4	0	tcp	http	SF	199	420	0	

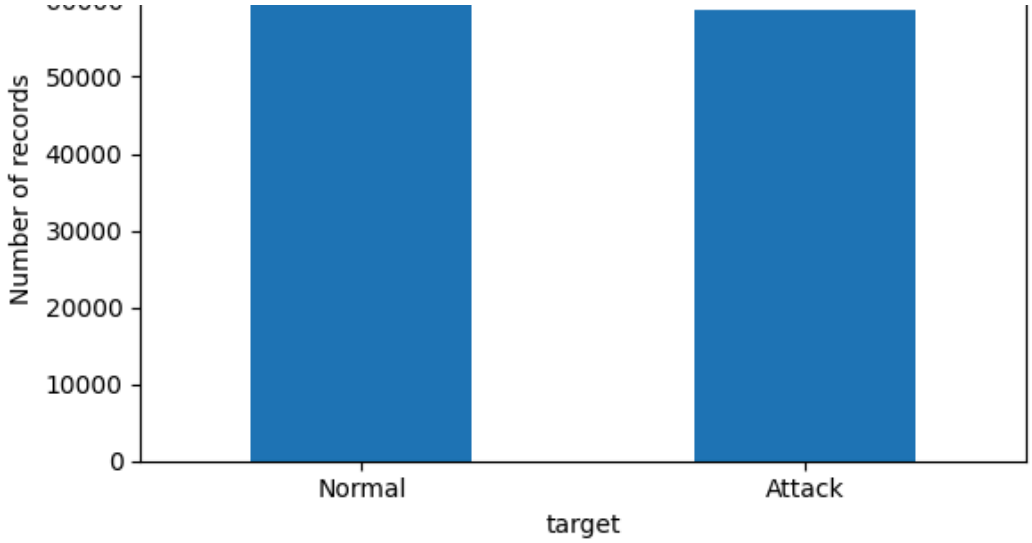
5 rows × 43 columns

```
Missing values: 0

Label distribution:
```

label	records
normal	67343
neptune	41214
satan	3633
ipsweep	3599
portsweep	2931
smurf	2646
nmap	1493
back	956
teardrop	892
warezclient	890
pod	201
guess_passwd	53
buffer_overflow	30
warezmaster	20
land	18
imap	11
rootkit	10
loadmodule	9
ftp_write	8
multihop	7
phf	4
perl	3
spy	2





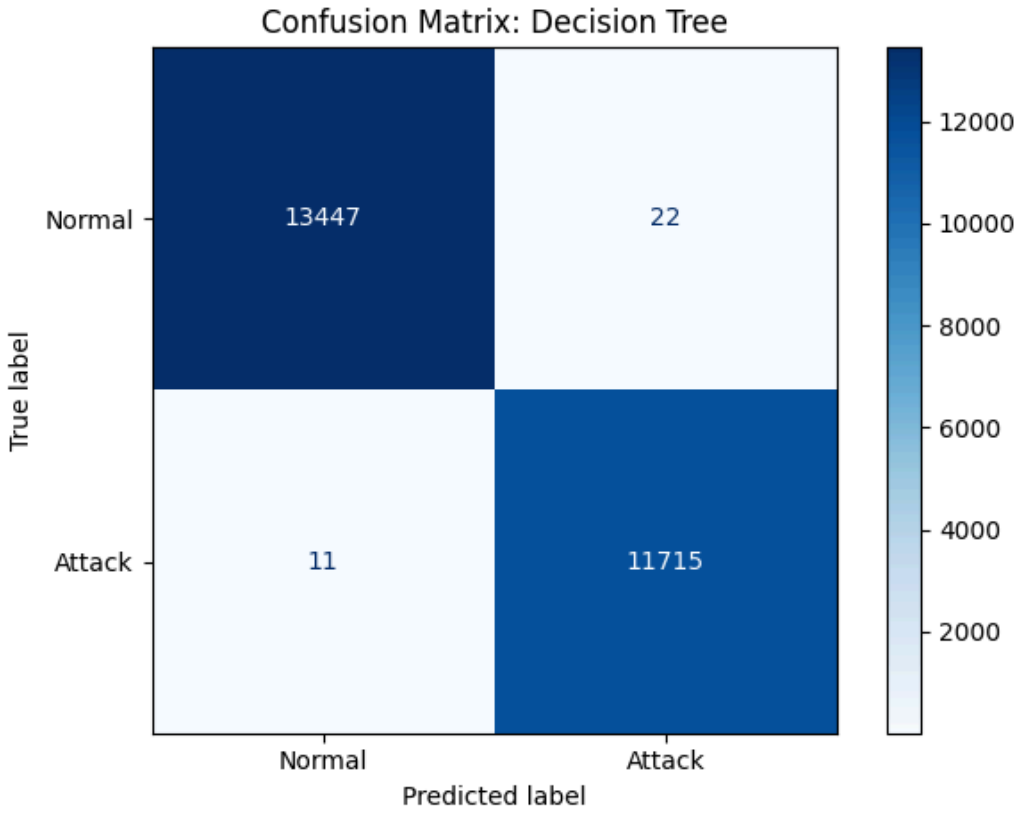

```
Number of input features: 41
Categorical features: ['protocol_type', 'service', 'flag']
Numeric features: 38

Decision Tree

              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00     13469
      Attack       1.00      1.00      1.00     11726

    accuracy                           1.00     25195
   macro avg       1.00      1.00      1.00     25195
weighted avg       1.00      1.00      1.00     25195
```

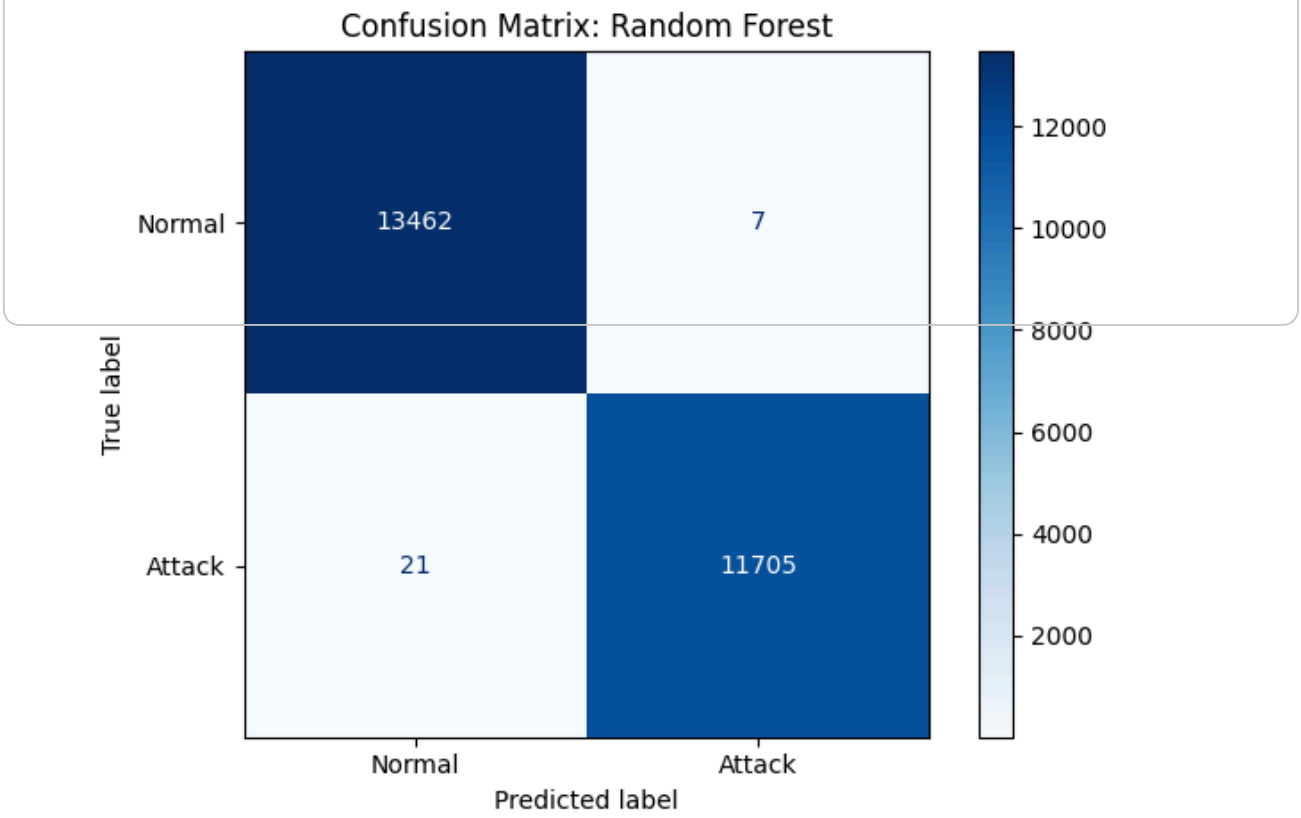



```
Random Forest

              precision    recall  f1-score   support

      Normal       1.00      1.00      1.00     13469
      Attack       1.00      1.00      1.00     11726
```

```
              accuracy                           1.00     25195
             macro avg       1.00      1.00      1.00     25195
          weighted avg       1.00      1.00      1.00     25195
```

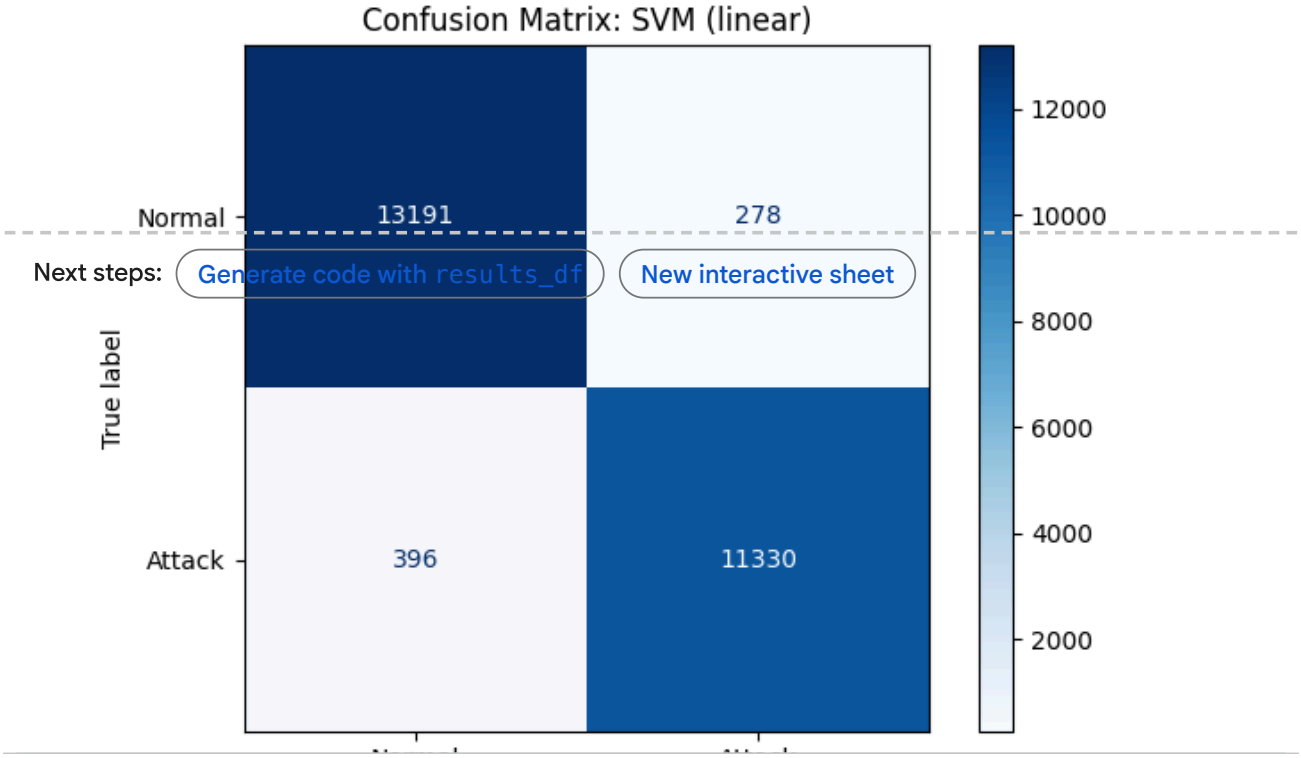


SVM (linear)

```
              precision    recall  f1-score   support

      Normal       0.97      0.98      0.98     13469
      Attack       0.98      0.97      0.97     11726

    accuracy                           0.97     25195
   macro avg       0.97      0.97      0.97     25195
weighted avg       0.97      0.97      0.97     25195
```



Next steps: Generate code with results_df New interactive sheet